

Milestone 2 Progress Summary

1. Current Status

We have had a working application with these features:

- + Start Default Simulation
- + Inject Vehicles from edges with customized colors, speed
- + Traffic lights manipulations. User can select, change colors, duration, or switch phase of the traffic lights
- + Stress testing. Users can inject many customized vehicles at once.
- + Statistic charts. Users can observe several statistics through the chart calculated in real time of the simulation. For e.g: congestion hotspot, average speed,...

2. Challenges

In the process of building this application, we stumbled a few challenges:

Data Fetch and Performance Optimization

- + Early development stages revealed inefficiencies in data handling due to redundant and frequent data fetching operations. This issue was resolved by separating static and dynamic data processing. Static objects are retrieved only once, while dynamic entities are updated per time step. Furthermore, we notice that when vehicles already exist within the simulation, their positions could be updated through coordinate recalculation from speed and time rather than repeated data fetching. This approach significantly improved performance, smoothness and overall look of the application.

Vehicle Injection:

- + Significant challenges were encountered during vehicle injection, particularly when routes were generated from arbitrarily selected edges. In many cases, the selected edges did not form valid routes or were appropriate for the selected vehicle type, leading to runtime errors and simulation instability. To mitigate this issue, the system now restricts users to only hollow and select valid and compatible edges. Additionally, comprehensive error handling mechanisms were implemented to prevent connection failures.

Multithreading and Data Synchronization:

- + The application employs three main concurrent threads responsible for simulation connection, statistical calculation, and rendering. Initially, this architecture led to data inconsistency and race conditions due to unsynchronized access to shared

resources. To address this, a thread-safe queue was introduced to manage simulation state versions. This solution ensures that all threads operate on consistent data, regardless of execution order or processing speed.

3. Future Features

We plan to include 2 more features in the up-coming milestone:

- + Statistical Report Exportation
- + Filtering (by color, speed,...)

4. Roles & Contribution

Team Member	Primary Role / Focus Area	Key Contributions and Modules
Hung	Core Execution, Threading & UI Integration	MainController (Orchestration), SimulationManager (High-level control), Thread Design (Using Executor for concurrency), MapInteractionHandler (Input handling), Primary FXML GUI (Structure and Layout).
Trang	Vehicle/Infrastructure Rendering & UI Logic	Renderer module (Rendering Logic for Vehicles, Lanes, and Junctions), Secondary FXML GUI elements, Pane Interaction Logic (e.g., Inject Vehicle, Stress Test triggers), ColorConverter utility.
Khoi	Vehicle Modeling & Data Analysis	VehicleManager (Life cycle and data fetching), VehicleClass and related subclasses, StatisticsManager (Data collection and aggregation), SubVehicle (Specific vehicle implementation/model), ChartWindow, Statistics on FXML.

Khang	Traffic Control, Rendering & Reporting	TrafficlightManager (Control logic), TrafficlightClass (Model), Traffic Light Rendering (Visualization of phases), FXML GUI (Trafficlight Control Pane), Logic for Setting Trafficlight Duration/Color, Switching Phases, ReportManager (Finalizing data output).
Khoa	Data Pipeline & Map Initialization	SimulationQueue (Producer-Consumer buffer), SimulationState (Data snapshot), SimulationManager (Specific implementation parts), Stress Testing Logic (Generating random vehicles and paths), MapManager (Edge/Lane structure initialization).